

UE5.7 AI 战斗调度算法说明

Director + AIPerception + BTTask + EQS 的蓝图实现逻辑

1. 目标

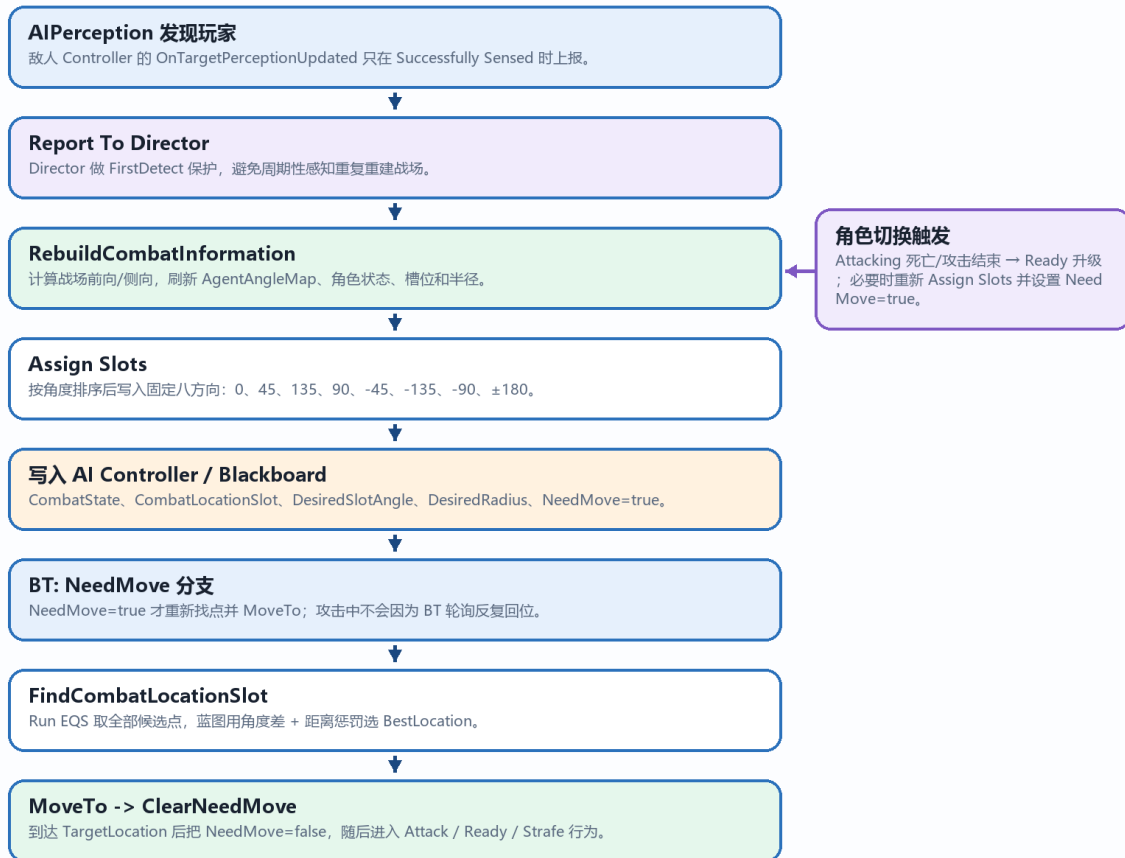
这个 AI 原型的核心目标是：当任意敌人发现玩家后，AI Director 统一接管战场，决定谁攻击、谁准备攻击、谁在外圈游走，并把每个敌人分配到战场八方向槽位。EQS 只提供可走候选点，最终位置由蓝图算法根据槽位角度与角色半径筛选。

2. 系统分工

模块	职责	关键数据
AIController	接收感知、写黑板、运行 SetFocus、执行角色状态切换。	AIState / CombatState / AttackTarget
BP_AIDirector	全局调度，计算战场坐标轴，排序 Agent，分配槽位和角色。	BattleForward / BattleSide / AgentAngleMap
Behavior Tree	用状态和 NeedMove 控制任务流，避免攻击后重复回位。	NeedMoveToCombatLocation / TargetLocation
EQS	生成并过滤候选点：投影、碰撞、可达、视线。	QueryLocation 数组
BTT_FindCombatLocationSlot	遍历 EQS 候选点，用角度差和距离惩罚选 BestLocation。	DesiredSlotAngle / DesiredRadius / BestScore

AI Director 战斗调度总流程

从感知玩家到分配槽位、选择点位、移动与清理 NeedMove 的完整闭环。



关键原则：UpdateNew 是“是否重建”的触发；NeedMove 是“是否继续移动到既定点”的锁存。

图 1：AI Director 战斗调度总流程

3. 战场坐标轴

- 战场中心使用玩家位置，也就是 AttackTarget Location。
- BattleForward 使用玩家到 FirstAgentSpottedPlayer 的 2D 归一化方向。这个方向定义为 Front_0。
- BattleSide 是 BattleForward 的垂直方向，用于区分左侧和右侧。当前调试结果约定为：正角在左，负角在右。
- 每个 AI 的角度通过 $\text{Atan2}(\text{SideDot}, \text{ForwardDot})$ 得到，范围是 -180 到 180。

战场坐标轴构建

以玩家为中心，用“第一个发现玩家的敌人”确定战场前方。后续所有角度都基于这套轴计算。

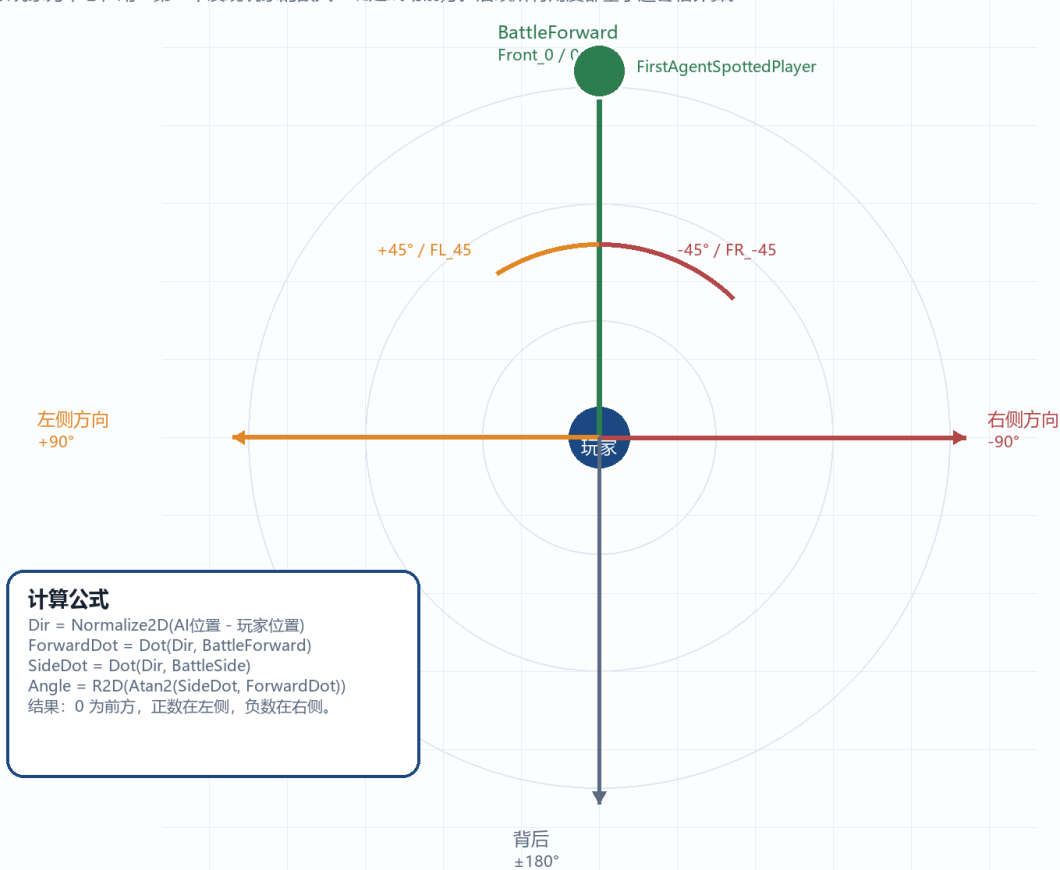


图 2: 战场轴、角度方向与 Dot/Atan2 的关系

4. 八边形槽位分配

Director 先计算每个敌人在战场坐标轴里的角度，然后调用 Assign Slots。输出的 OrderedAgents 与 OrderedSlotAngles 使用同一个 ArrayIndex，对应到蓝图枚举 CombatLocationSlot。

八边形敌人位置分配

排序函数输出固定槽位顺序，蓝图用同一个 ArrayIndex 取 Agent、SlotEnum、DesiredSlotAngle。

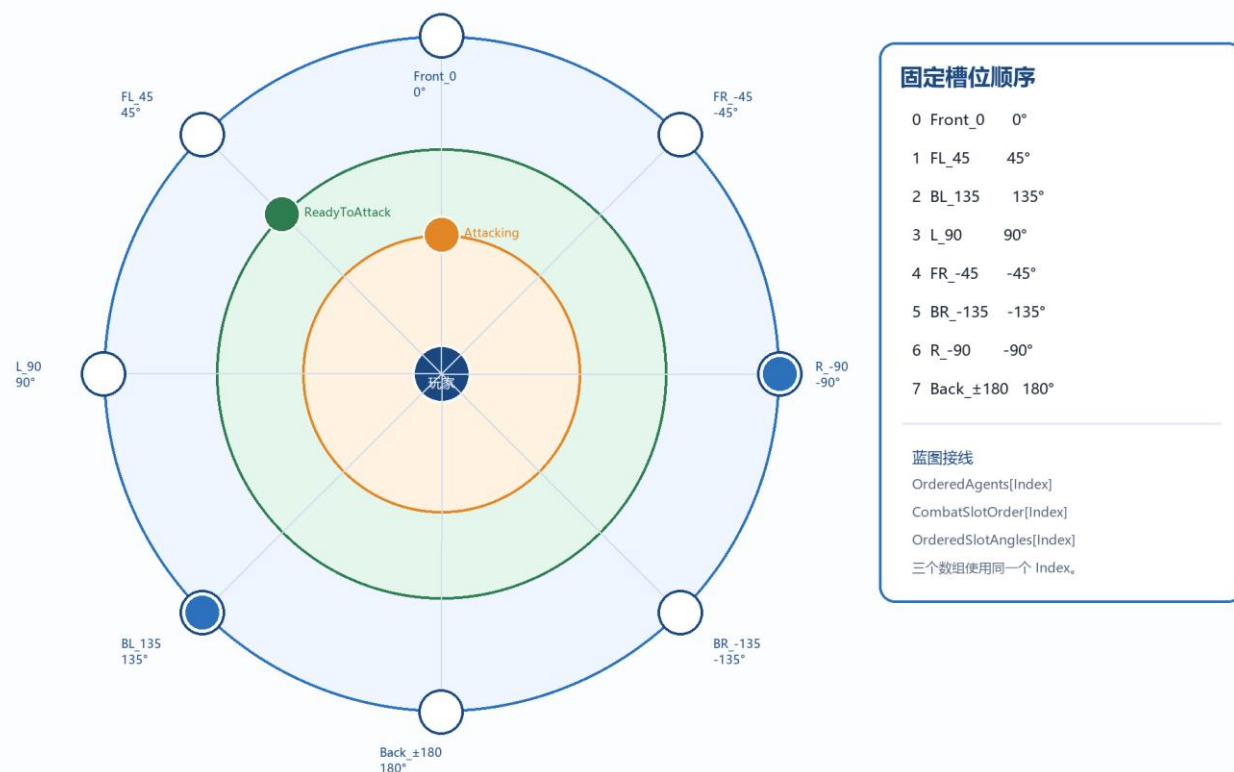


图 3：八方向槽位、角色半径与蓝图数组对应关系

5. 槽位与角色规则

CombatState	选择规则	DesiredRadius	行为
Attacking	第一个发现玩家的敌人，或 Ready 升级后的敌人。	300	内圈接近并攻击。
ReadyToAttack	除 Attacking 外，距离玩家最近的敌人。	600	第二圈待命，当前攻击者死亡/结束后升级。
Strafe	其余敌人。	1000	外圈保持包围和侧移。

6. EQS 候选点打分

EQS 不直接决定敌人应该去哪一侧。EQS 只生成有效点，蓝图任务再按每个 AI 的 DesiredSlotAngle 与 DesiredRadius 做最终选择。

推荐公式：

- $\Delta Angle = AbsDeltaAngle(QueryAngle, DesiredSlotAngle)$
- $DistancePenalty = Abs(Distance(PlayerLocation, QueryLocation) - DesiredRadius)$
- $LocationScore = \Delta Angle * AngleWeight + DistancePenalty$
- LocationScore 越小，说明这个点越符合目标方向和目标圈层。

EQS 候选点选择与移动锁存

EQS 只负责生成可走点；蓝图任务负责打分，NeedMove 负责防止攻击后反复回位。



图 4：EQS 候选点评分与 NeedMove 锁存流程

7. NeedMove 锁存规则

NeedMoveToCombatLocation 不应该跟 UpdateNew 直接绑定。UpdateNew 只表示是否需要重建战场信息；NeedMove 表示这个 AI 是否还需要移动到已分配的目标点。

- 设置为 true：首次进入战斗、玩家距离战场中心超过阈值、角色切换、槽位重分配。
- 设置为 false：只在 MoveTo TargetLocation 成功之后，由 BTT_ClearNeedMove 清理。
- 不要在玩家停止移动或 UpdateNew=false 时清 false，否则敌人会在还没到点时停止。

8. 蓝图落地顺序

- AI Perception 成功感知玩家 → Report To Director。

- Director 首次检测保护: IsFirstDetected=false 才执行完整构建。
- RebuildCombatInformation: 计算轴、角度、排序、槽位、角色、半径。
- 给每个 AIController 写 DesiredSlotAngle、CombatLocationSlot、DesiredRadius、NeedMove=true。
- BT 的 NeedMove 分支运行 BTT_FindCombatLocationSlot → MoveTo → BTT_ClearNeedMove。
- 攻击、准备攻击、Strafe 分支只读 CombatState, 不在每次攻击结束后强制重新找点。